

LAB: Digital Image Processing

Objective: To understand the image histograms representation & Compute/visualize histograms using OpenCV and Matplotlib.
Perform histogram equalization to enhance image contrast.

What is a Histogram?

A histogram in image processing shows how pixel intensities are distributed in an image.

X-axis → pixel intensity values (0–255 for 8-bit images)

Y-axis → frequency of each intensity

Why Use Histogram?

- To understand brightness and contrast.
- To identify low-contrast or overexposed images.
- To enhance images using histogram equalization

Step 1: Import Required Libraries

```
[1]: import cv2  
import numpy as np  
import matplotlib.pyplot as plt
```

Step 2: Read and Display an Image)

```
[3]: # Read an image in grayscale
img = cv2.imread('lena.jpg', 0)

# Display image
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.show()
```

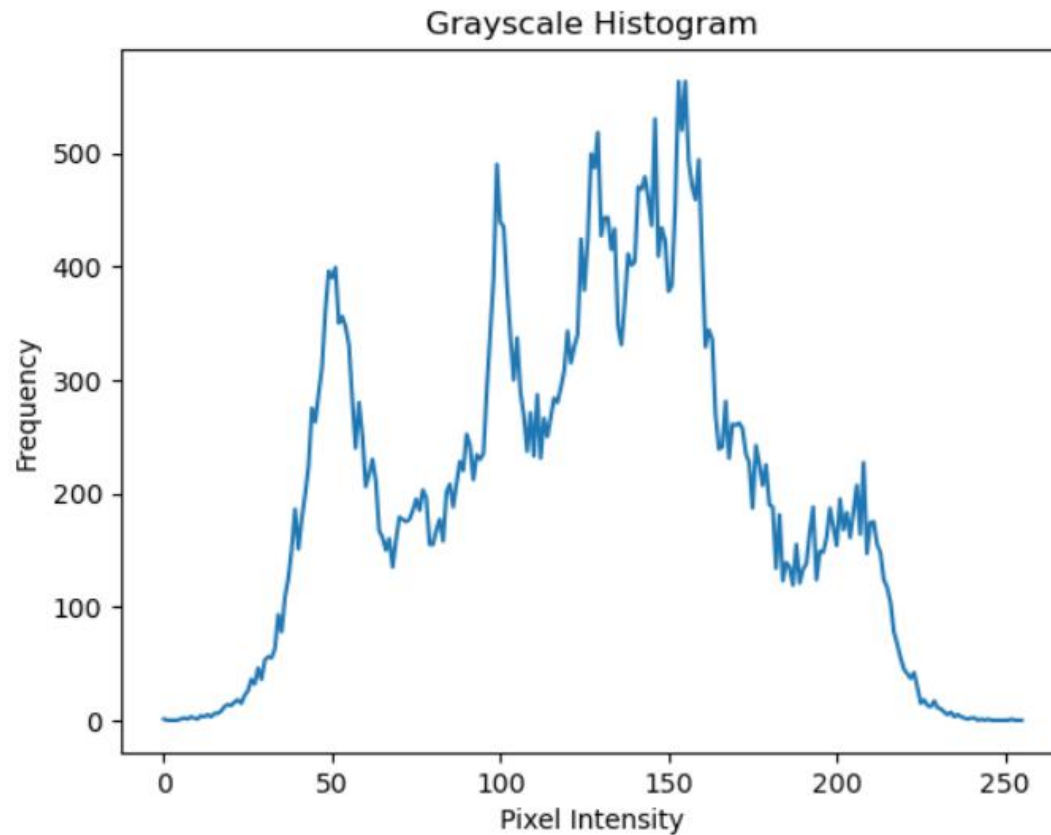
Original Image



Step 3: Compute Histogram Using OpenCV

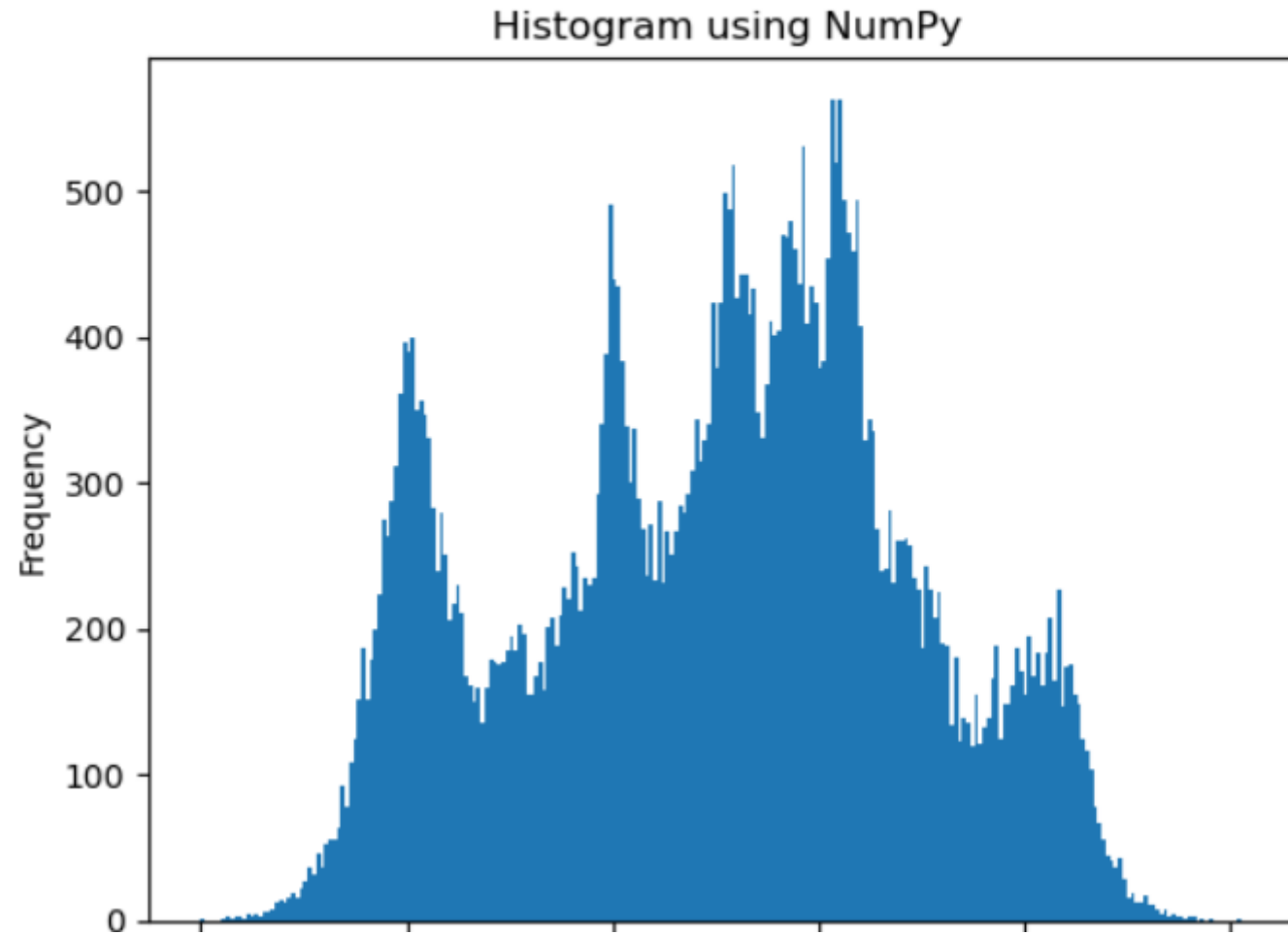
```
[4]: # Compute histogram
hist = cv2.calcHist([img], [0], None, [256], [0,256])

# Plot histogram
plt.plot(hist)
plt.title('Grayscale Histogram')
plt.xlabel('Pixel Intensity')
plt.ylabel('Frequency')
plt.show()
```



Step 4: Compute Histogram Using NumPy and Matplotlib

```
[5]: plt.hist(img.ravel(), bins=256, range=(0,256))  
plt.title('Histogram using NumPy')  
plt.xlabel('Pixel Intensity')  
plt.ylabel('Frequency')  
plt.show()
```



Parameters

- `img.ravel()` flattens the 2D grayscale image into a **1D array** — it simply lists all pixel intensity values (from 0 to 255) in a single row.
- `bins=256` divides the range of intensity values into **256 equal intervals**, one for each possible grayscale level.
- `range=(0,256)` specifies the **intensity range** to be plotted (0 = black, 255 = white).
- `plt.hist()` automatically counts how many pixels fall into each bin and displays this as a **bar graph (histogram)**.

Task

Change the bin size as shown above.

Observe how the histogram shape changes.

Write your observation:

Does fewer bins make the histogram smoother or more detailed?

Student Task

Load a different image of your choice.

Plot its histogram using both:

`cv2.calcHist()`

`plt.hist()`

Observe and describe what type of contrast it has (low / high / normal).

Task

Try different intensity ranges: (0,256), (0,100), (100,256).

Observe what part of the image intensities are emphasized.

Analyze:

Which range highlights the darker regions?

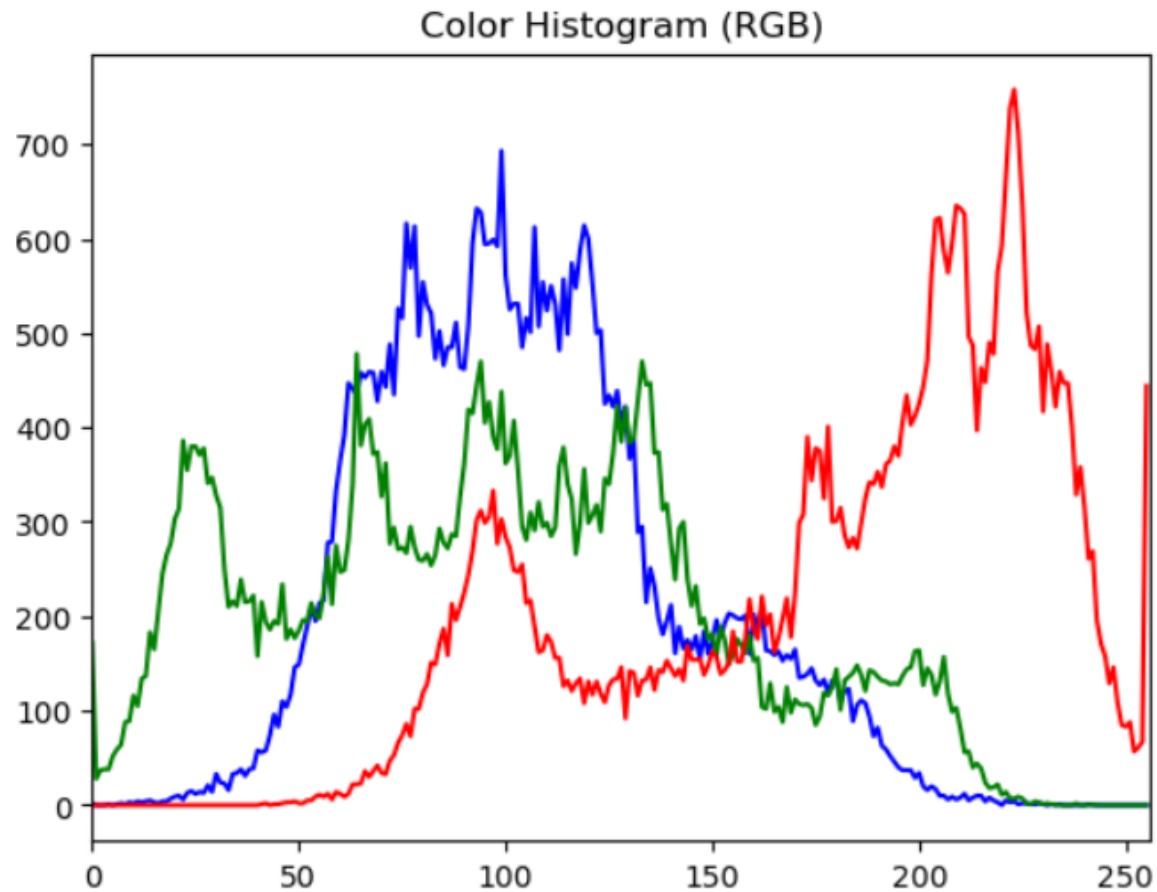
Which range focuses on bright areas?

Color Image Histogram

```
[7]: img_color = cv2.imread('lena.jpg')

colors = ('b','g','r')
for i, col in enumerate(colors):
    hist = cv2.calcHist([img_color], [i], None, [256], [0,256])
    plt.plot(hist, color=col)
    plt.xlim([0,256])

plt.title('Color Histogram (RGB)')
plt.show()
```



Histogram Equalization

Histogram equalization improves image contrast by redistributing intensity values.

The transformation function is based on the cumulative distribution function (CDF):

$$s_k = (L - 1) \sum_{j=0}^k p(r_j)$$

where $p(r_j) = \frac{n_j}{n}$ is the probability of each intensity.

Step 1: Apply Histogram Equalization

```
[8]: # Equalize histogram
equalized_img = cv2.equalizeHist(img)

# Display original and equalized images
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(equalized_img, cmap='gray')
plt.title('Equalized Image')
plt.axis('off')

plt.show()
```

Original Image



Equalized Image



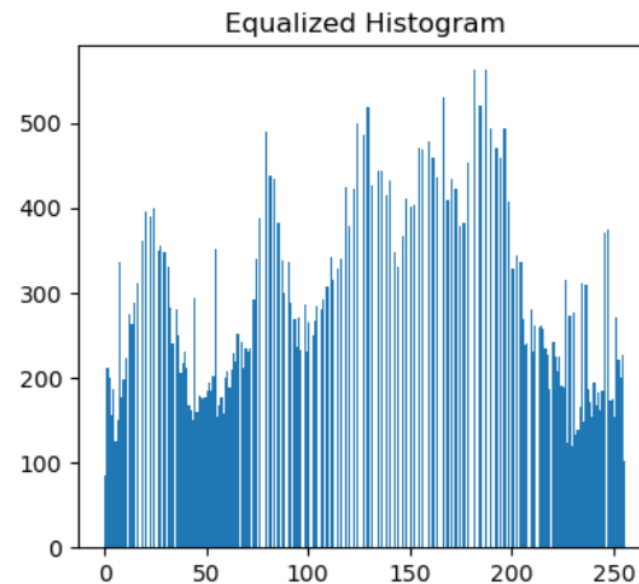
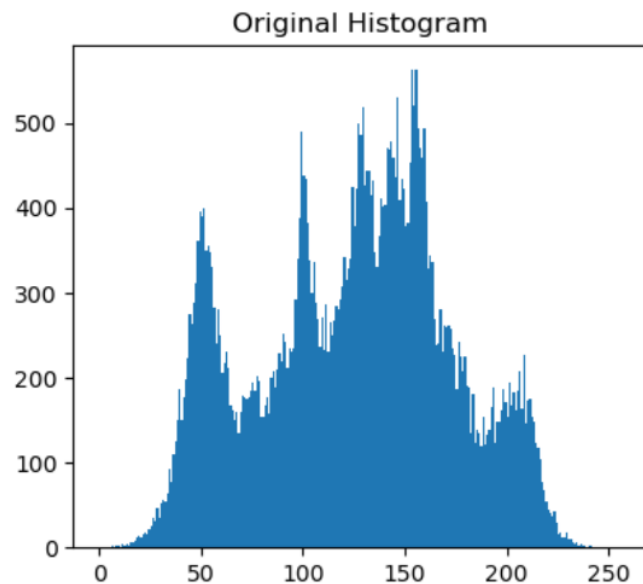
Step 2: Compare Histograms

```
[9]: plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.hist(img.ravel(), 256, [0,256])
plt.title('Original Histogram')

plt.subplot(1,2,2)
plt.hist(equalized_img.ravel(), 256, [0,256])
plt.title('Equalized Histogram')

plt.show()
```



Task

Now, modify the given code to experiment with different histogram parameters and observe how they affect both visual and numerical outcomes.

Your goal is to analyze how the histogram changes when you:

- Adjust the number of bins (e.g., 32, 64, 128, 512).
- Modify the intensity range (e.g., [0,128], [50,200], [100,256]).
- Apply equalization to different types of images (dark, bright, or low contrast).
- Compare how equalization redistributes pixel intensities across the histogram.?